

Healthcare Appointment & Follow-up Manager

Database Schema Design • API Design • Code Structure

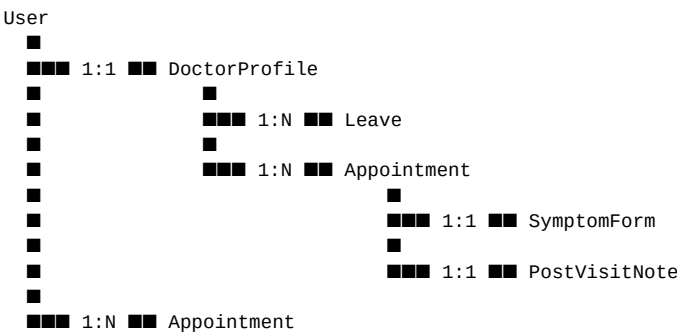
Project Documentation for Submission

Project basis: This document is based on the supplied project brief and the project repository reviewed for the submission. It focuses on the database schema, API design, architecture, and code organization.

1. Database Schema Design

The application uses SQLite with Django ORM. The database is organized around the healthcare workflow: User → Doctor Profile → Appointment → Consultation, with supporting entities for doctor leave and notifications. Role-based authentication supports patient, doctor, and admin users.

1.1 Entity Relationship Overview



1.2 User

Field	Type	Description
id	Integer PK	Unique user identifier
username	String	Django authentication username
email	Email	User email address
role	Choice	patient / doctor / admin
phone_number	String	User contact number
date_of_birth	Date	User date of birth

1.3 DoctorProfile

Field	Type	Description
id	Integer PK	Doctor profile identifier
user_id	OneToOne FK	Associated User
specialisation	String	Medical specialization
working_hours_start	Time	Start of consultation hours
working_hours_end	Time	End of consultation hours
slot_duration_minutes	Integer	Duration of each slot
working_days	String	Configured working weekdays
bio	Text	Doctor description
is_active	Boolean	Whether doctor is available for booking

1.4 Leave

Field	Type	Description
id	Integer PK	Leave identifier
doctor_id	FK	Doctor taking leave
date	Date	Leave date
reason	String	Reason for leave
created_at	DateTime	Record creation timestamp

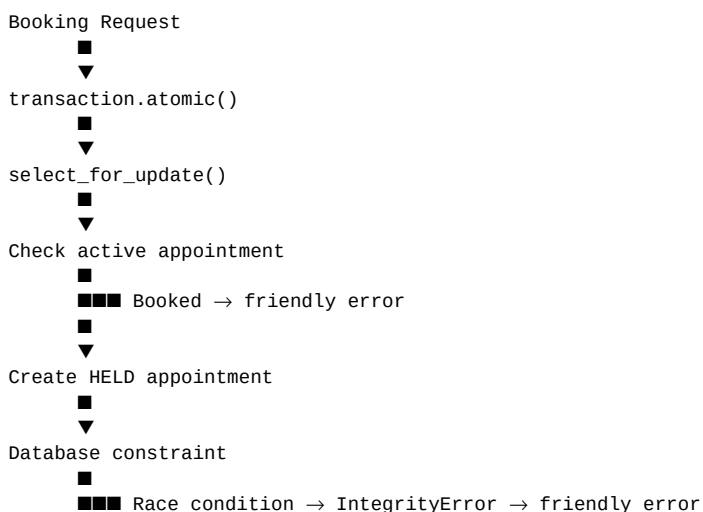
Database rule: (doctor, date) is unique, preventing duplicate leave entries for the same doctor and date.

1.5 Appointment

Field	Type	Description
id	Integer PK	Appointment identifier
patient_id	FK User	Patient making the appointment
doctor_id	FK DoctorProfile	Doctor being consulted
date	Date	Appointment date
start_time	Time	Start time
end_time	Time	End time
status	Choice	held / confirmed / cancelled / completed / leave_cancelled
hold_expires_at	DateTime	Temporary slot hold expiration
google_event_id_patient	String	Patient calendar event ID
google_event_id_doctor	String	Doctor calendar event ID
created_at	DateTime	Creation timestamp
updated_at	DateTime	Last update timestamp

1.6 Double-Booking Prevention

Appointment booking is protected at both application and database levels. The booking flow uses a database transaction and row locking while checking an existing appointment. The database also uses a conditional uniqueness rule for active slots (held/confirmed). If a race condition still reaches the database constraint, `IntegrityError` is handled and converted into a user-friendly slot-unavailable response.



1.7 SymptomForm

Field	Type	Description
id	Integer PK	Record identifier
appointment_id	OneToOne FK	Associated appointment
symptoms_text	Text	Patient's symptoms
urgency_level	Choice/String	Low / Medium / High
chief_complaint	String	Main complaint
suggested_questions	JSON	AI-generated questions
llm_status	Choice	pending / success / failed
raw_llm_response	JSON	Raw AI response for debugging

1.8 PostVisitNote

Field	Type	Description
id	Integer PK	Record identifier
appointment_id	OneToOne FK	Associated appointment
clinical_notes	Text	Doctor's raw clinical notes
prescription_text	Text	Doctor's prescription
medications	JSON	Medication list with dosage/frequency/duration
patient_summary	Text	AI-generated patient-friendly summary
follow_up_steps	Text	AI-generated follow-up instructions
llm_status	Choice	pending / success / failed

2. API Design

The application follows Django's server-rendered architecture, with HTML templates for the primary interface and a JSON endpoint for dynamic slot availability. JavaScript fetch() is used where asynchronous slot updates are needed.

2.1 Available Slots API

Endpoint: GET /appointments/api/slots/

Query parameters: doctor_id, date

```
GET /appointments/api/slots/?doctor_id=2&date=2026-08-25
```

Response:

```
{
  "doctor_id": 2,
  "date": "2026-08-25",
  "slots": [
    "09:00",
    "09:30",
    "10:00",
    "10:30"
  ]
}
```

The endpoint validates the doctor and date, calculates working slots, removes occupied/unavailable slots, and returns the remaining slots as JSON.

2.2 API Error Responses

Condition	HTTP	Response
Missing doctor_id/date	400	doctor_id and date query parameters are required
Invalid doctor	404	Doctor not found
Invalid date	400	Invalid date format. Use YYYY-MM-DD

2.3 Application Routes

Method	Endpoint	Purpose
GET/POST	/accounts/register/	Patient registration
GET	/accounts/login/	Login
POST	/accounts/logout/	Logout
GET	/accounts/dashboard/	Role-based dashboard routing
GET	/appointments/api/slots/	Available appointment slots
GET	/appointments/doctors/	Doctor search
GET	/appointments/doctors/<id>/	Doctor details and slots
POST	/appointments/book/	Hold/book an appointment
GET	/appointments/mine/	Patient appointments
GET	/appointments/doctor/	Doctor appointments
POST	/appointments/<id>/cancel/	Cancel appointment

2.4 Booking API / Business Flow

1. GET /appointments/api/slots/
→ Fetch currently available slots
2. POST /appointments/book/
→ Patient selects doctor/date/start time
3. hold_slot()

- Appointment is created as HELD
 - hold_expires_at is set
4. Patient submits symptom form
 - LLM generates pre-visit information
 5. Appointment becomes CONFIRMED
 - Consultation workflow can proceed

3. Code Structure

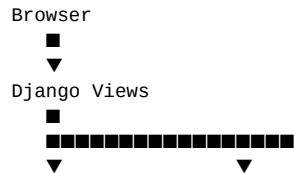
The project uses a modular Django architecture. Each application has a focused responsibility, reducing coupling and keeping authentication, scheduling, consultations, and integrations separate.

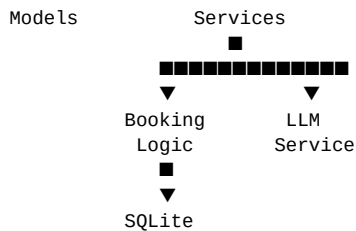
```
Clinic_AI_Healthcare_Appointment/
├── accounts/
│   ├── models.py
│   ├── views.py
│   ├── forms.py
│   ├── decorators.py
│   ├── urls.py
│   └── admin.py
├── doctors/
│   ├── models.py
│   ├── views.py
│   └── admin.py
├── appointments/
│   ├── models.py
│   ├── views.py
│   ├── services.py
│   └── urls.py
├── consultations/
│   ├── models.py
│   ├── views.py
│   └── llm_service.py
├── notifications_app/
├── clinic_project/
│   ├── settings.py
│   ├── urls.py
├── templates/
├── static/
├── manage.py
├── requirements.txt
└── db.sqlite3
```

3.1 Application Responsibilities

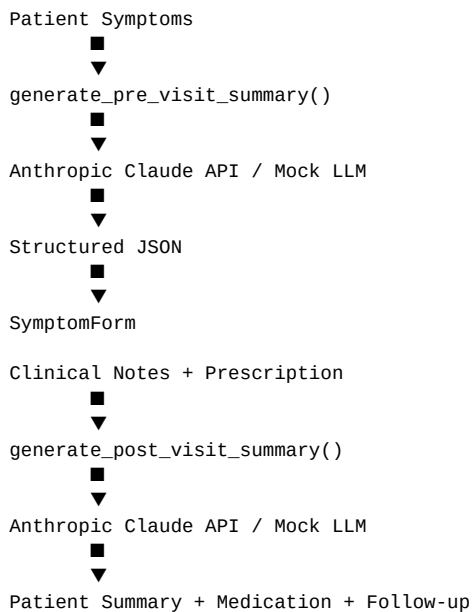
Django App	Responsibilities
accounts	Custom User model, authentication, roles, registration, dashboard routing, access control
doctors	Doctor profiles, specialization, working hours/days, slot duration, leave
appointments	Appointment model, availability, booking, slot holding, cancellation, concurrency protection
consultations	Symptoms, AI pre-visit summary, clinical notes, prescriptions, post-visit summary
notifications_app	Notification/integration layer intended for email, calendar, and background processing

3.2 Separation of Concerns





3.3 AI Service Architecture



4. Design Rationale

SQLite: Suitable for the development/demo environment and specified in the project requirements.

Separate Django apps: Keeps authentication, doctors, appointments, consultations and notifications independently maintainable.

JSONField: Useful for semi-structured AI output such as suggested questions and multiple medication entries.

Service layer: Keeps concurrency-sensitive booking logic out of views and makes it reusable/testable.

Database constraints: Provides a final protection against concurrent double-booking beyond application-level checks.

5. Implementation Status / Submission Note

The repository was reviewed against the supplied project brief. The following distinction should be maintained in the submission: documentation should describe implemented functionality accurately and should not claim incomplete integration features as fully implemented.

Area	Observed Status
Custom User / roles	Implemented
DoctorProfile	Implemented
Leave model	Implemented
Appointment model	Implemented
Double-booking protection	Implemented
Slot holding	Implemented
SymptomForm	Implemented
PostVisitNote	Implemented
LLM / Mock LLM service	Implemented
Dynamic slot JSON API	Implemented
notifications_app	Present, but incomplete in reviewed repository
EmailLog / MedicationReminder	Not currently evident in reviewed repository
Google Calendar persistence/service	Not currently evident in reviewed repository

6. Conclusion

The database and API architecture is centered on a reliable appointment lifecycle. The strongest design element is the multi-layer double-booking protection using transactions, row locking, and a database constraint. The modular Django structure also separates domain responsibilities cleanly, while the LLM service isolates AI-specific behavior from the consultation workflow.

This structure directly addresses the evaluation areas of database schema design, API design, code structure, concurrency handling, and LLM integration described in the project brief.